

Parallel OpenMP and CUDA Implementations of Count Gliders, Histogram, and Emboss

COM4521/COM6521 Parallel Computing with GPUs

Abstract

This report describes and evaluates OpenMP and CUDA implementations of three serial reference algorithms: counting Game of Life gliders, constructing an integer histogram, and applying a greyscale emboss stencil to an RGB image. The implementations were designed to preserve the exact behaviour of the supplied CPU reference while exposing the main parallel patterns in the workload: independent window classification, many-to-few accumulation, and stencil-style image processing. OpenMP was used to exploit multicore CPU parallelism through reductions, per-thread aggregation, and collapsed two-dimensional loops. CUDA was used to map work to GPU threads with grid-stride loops, block-level reduction, shared-memory histograms, atomics, and coalesced output writes. Performance was evaluated in the Release configuration using the supplied benchmark mode, which averages 100 runs per input. On an Intel i9-14900HX and NVIDIA RTX 4060 Laptop GPU, all prepared correctness and benchmark cases passed validation. The strongest acceleration was observed for large Count Gliders and Emboss inputs, while Histogram showed more limited speedup because its update pattern is dominated by contention and small output size.

1 Methodology

The report evaluates the submitted implementations against the supplied CPU reference rather than against an independently rewritten specification. This is important because the marking harness validates the OpenMP and CUDA outputs by comparing them with the serial implementation. For each algorithm, the implementation was therefore first checked for functional equivalence and then benchmarked in the same executable. The final benchmark run used the Visual Studio Release x64 configuration, built with CUDA Toolkit 12.6. The test machine used an Intel(R) Core(TM) i9-14900HX CPU and an NVIDIA GeForce RTX 4060 Laptop GPU with 8188 MiB of device memory and driver version 595.79. The Release build completed with zero warnings and zero errors.

Timing data was collected using the assignment harness with the `--bench` flag. This repeats the selected implementation 100 times and reports the average runtime, which reduces the effect of one-off launch, scheduling, and operating system noise. Count Gliders was measured on the provided `cg_16_in.png` image and on deterministic tiled glider images of size 256^2 , 1024^2 , and 2048^2 . Histogram was measured with pseudo-random integer inputs of length 4096, 1,000,000, and 16,000,000, using bin widths 10, 17, and 64. Emboss was measured on deterministic RGB images of size 256^2 , 1024^2 , and 2048^2 . Across this matrix, 48 benchmark cases were run and all passed the harness validation.

The main performance evidence is benchmark-based, supported by CUDA profiling where possible. Nsight Compute was attempted for the CUDA kernels, but the local Windows/NVIDIA configuration blocked hardware performance counters with `ERR_NVGPUCTRPERM`. To avoid making unsupported claims about occupancy or memory throughput, the report instead uses Nsight Systems CUDA traces for kernel timing evidence. The traces show representative kernel durations of 126.430 microseconds for `countGlidersKernel` on a 2048^2 tiled image, 259.227 microseconds for `histogramKernel` on 16,000,000 integers with bin width 10, and 206.588 microseconds for `embossKernel` on a 2048^2 RGB image. These values are used as supporting evidence for launch and kernel behaviour, while the broader scaling claims are drawn from the 100-run benchmark results.

Table 1: Benchmark and profiling evidence used in the report.

Evidence type	Inputs	Purpose
Correctness/build Count Gliders	Release x64, 48 benchmark cases <code>cg_16.in.png</code> , 256^2 – 2048^2 tiled images	Confirm reference-equivalent outputs Window scan scaling
Histogram	4096–16,000,000 integers, bin widths 10, 17, 64	Contention and bin-width effects
Emboss CUDA profiling	256^2 – 2048^2 RGB images Nsight Systems kernel traces	Stencil scaling and image throughput Kernel timing evidence

2 OpenMP Implementation

The OpenMP implementation targets the shared-memory CPU case, where the main cost is dividing independent work across cores while avoiding unnecessary synchronisation. The implementation keeps the supplied function interfaces and the C compilation model, so the code remains compatible with the Visual Studio marking environment. Static scheduling was used throughout because the work per loop iteration is regular: each glider window, histogram input value, or emboss output pixel performs a fixed amount of local computation. This avoids the extra runtime cost of dynamic scheduling while still distributing the iteration space across the available CPU threads.

For Count Gliders, each possible 3×3 window is independent. The OpenMP version therefore flattens the two-dimensional window space into a single loop and applies a reduction to the total count. Rather than reproducing the reference implementation’s repeated reflection and rotation logic inside every iteration, each window is encoded as a 9-bit mask and compared against the sixteen valid glider masks. This is still an exact match over all nine cells, so it preserves the reference behaviour, including overlapping or adjacent gliders that happen to form valid windows. The reduction is the only shared update, which makes the parallel structure simple and avoids locking for each detected match. The main limitation is that small images cannot amortise the cost of starting the OpenMP region; this is visible on the tiny supplied sample, where parallel overhead is larger than the useful work.

Histogram has a different parallel pattern because many input elements update a small number of output bins. A direct shared output array with an atomic increment for every input value would be correct, but it would also create heavy contention, especially because all numbers are in the range $[0, 255]$. The implementation instead allocates a private histogram for each OpenMP thread. Each thread updates only its own bins during the input scan, and a second parallel loop combines the per-thread bins into the final output. This converts the highly contended update phase into local memory accesses followed by a small deterministic merge. The output length follows the CPU reference, while the internal storage covers the possible edge bin so that values equal to 255 are handled safely for bin widths that do not divide the range evenly. The trade-off is that the method performs extra initialisation and a merge step, so it is not beneficial for very short arrays where the serial reference already completes in microseconds.

For Emboss, the output image is two pixels smaller in each dimension and each output pixel depends only on a fixed 3×3 neighbourhood of the input. This is a stencil-style gather operation with no dependency between output pixels, so the implementation uses a collapsed two-dimensional OpenMP loop over the output coordinates. Each iteration converts the nine RGB pixels to greyscale, applies the emboss kernel, adds 128, clamps to the valid byte range, and writes one output byte. The collapsed loop exposes enough parallelism for larger images and keeps writes contiguous across rows. The design deliberately avoids a more complex shared tile structure on the CPU because the stencil is small and the code is limited by simple arithmetic and cache-friendly neighbouring loads. Its main limitation is that neighbouring output pixels reload

overlapping input pixels, but the added complexity of manual tiling would be unlikely to justify itself for this small kernel in an OpenMP C implementation.

3 CUDA Implementation

The CUDA implementation follows the same reference-equivalence principle as the OpenMP version, but the design decisions are different because the GPU favours large numbers of lightweight threads and explicit memory movement. Each CUDA wrapper allocates device memory for the current call, copies the required host input to the device, launches one or more kernels, copies the result back to host memory, and then frees every allocation before returning. This avoids hidden state between benchmark repetitions and makes memory ownership clear. All CUDA API calls and kernel launches are checked through the supplied error-checking macros, which is important because asynchronous failures can otherwise appear far away from the operation that caused them. The implementation deliberately avoids Thrust and CUB, including for Histogram, so that the submitted code demonstrates the parallel patterns directly.

For Count Gliders, the GPU kernel maps one logical task to each 3×3 candidate window, with a grid-stride loop so that the same kernel configuration can cover both small and large images. Each thread encodes its assigned window as a 9-bit mask and compares it with a constant set of sixteen valid glider masks. This removes the repeated rotation and reflection work from the serial reference while preserving the same exact window-level definition of a match. A thread accumulates a local count across all windows reached by its grid-stride loop. The block then reduces these local counts in shared memory using a sequential halving pattern, and only one thread per block performs a global `atomicAdd`. This is preferable to one global atomic per matching window because matches can be frequent in the tiled benchmark images, and global atomics would serialize more updates. The remaining global atomic is small: it is proportional to the number of blocks, not the number of windows.

Histogram is the algorithm with the most obvious contention. The input range is only $[0, 255]$, so even very large arrays update a small number of bins. A simple CUDA implementation using `atomicAdd` directly on global output bins would be correct but would force many blocks to contend for the same global addresses. The implementation therefore uses a shared-memory histogram per block. Threads first cooperatively clear the block-local bins, then scan the input with a grid-stride loop. Each input value increments a shared-memory bin with an atomic operation, and after all threads in the block finish, the shared bins are flushed to the global histogram with one global atomic per bin per block. This does not remove contention entirely, because the algorithm is inherently a many-to-few accumulation, but it changes the hottest update phase from global memory to lower-latency shared memory. It also matches the module’s parallel-pattern guidance: use local aggregation to reduce global synchronisation pressure. The output length follows the CPU reference, and internal storage includes the possible edge bin so that values at 255 are handled safely for non-dividing bin widths.

For Emboss, the CUDA mapping is a two-dimensional image kernel. A 16×16 thread block is used so that each block covers a rectangular tile of output pixels and the block size remains a multiple of the 32-thread warp size. Each thread computes one output pixel: it reads the corresponding 3×3 RGB neighbourhood, converts each RGB triplet to greyscale, applies the fixed emboss stencil, adds 128, clamps to $[0, 255]$, and writes one output byte. The write pattern is coalesced along each output row because adjacent threads in the x dimension write adjacent bytes. The input reads overlap between neighbouring output pixels, but the stencil is very small and the implementation avoids shared-memory tiling because the extra indexing, halo loading, and synchronisation would increase complexity for a limited expected benefit. In contrast to Histogram, there are no inter-thread updates or reductions in Emboss, so the GPU version is mostly limited by memory movement, arithmetic per output pixel, and launch overhead on smaller images.

Across the CUDA implementations, the main design trade-off is between simple one-call wrappers and more aggressive reuse or batching. Reusing device buffers across calls could reduce allocation overhead during benchmarking, and streams could overlap independent transfers and kernels. However, the harness calls one algorithm at a time and validates each result synchronously, so persistent buffers or stream pipelines would add state and failure modes without changing the algorithmic parallelism being assessed. The chosen approach is therefore intentionally explicit: each call owns its memory, launches kernels with enough parallelism for large inputs, and returns a host result that can be compared directly with the CPU reference.

4 Results and Discussion

All 48 benchmark cases passed validation, so the performance comparison is between implementations that produced the same outputs as the CPU reference. Figure 1 shows the average end-to-end runtime reported by the harness. The strongest scaling appears in Count Gliders. On the 2048^2 tiled image, the CPU reference took 710.702 ms, compared with 6.501 ms for OpenMP and 1.250 ms for CUDA. This corresponds to speedups of about $109\times$ and $569\times$, respectively, as shown in Figure 2. The result is consistent with the algorithm structure: each candidate window can be classified independently, and the optimized implementations replace repeated transformation checks with compact mask matching. The small `cg_16.in.png` case behaves differently: the CPU takes only 0.054 ms, while OpenMP and CUDA take 0.222 ms and 0.441 ms. For such a tiny input, parallel setup, kernel launch, allocation, and copy overhead dominate the actual window work.

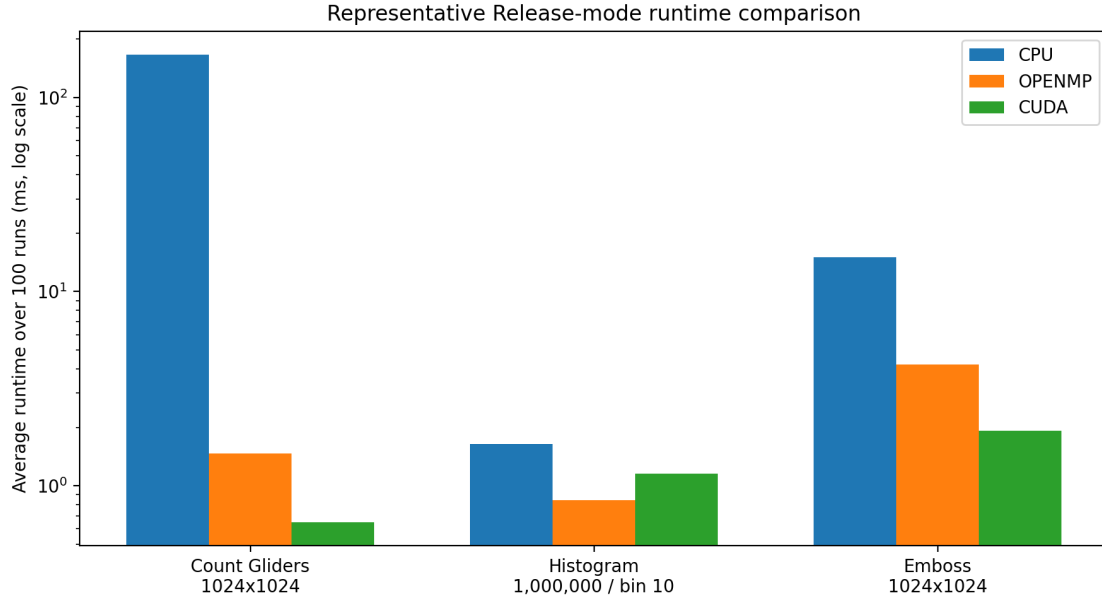


Figure 1: Average harness runtime for CPU, OpenMP, and CUDA across the benchmark inputs.

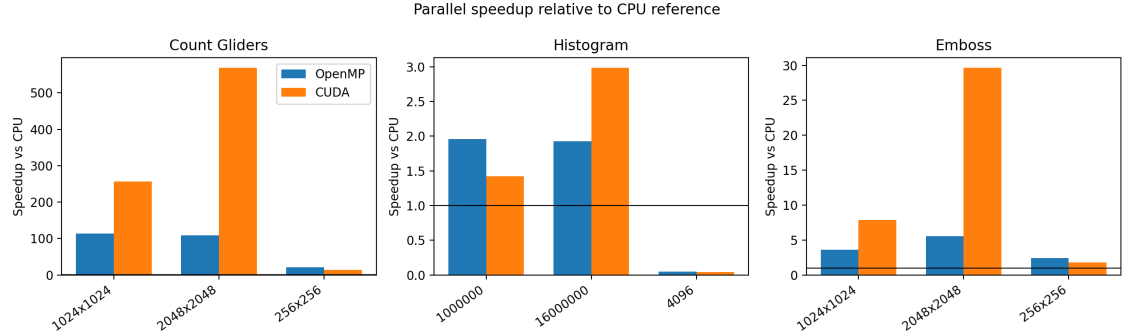


Figure 2: Speedup of OpenMP and CUDA relative to the supplied CPU reference.

Emboss also scales well because each output pixel is independent. On the 2048^2 RGB input, runtime decreased from 104.178 ms on the CPU to 18.814 ms with OpenMP and 3.515 ms with CUDA. The OpenMP speedup is lower than Count Gliders because each output pixel performs only a small fixed stencil computation, so the CPU reference already benefits from cache locality and sequential memory access. CUDA gains more because a large image exposes millions of independent output pixels and the writes are coalesced along rows. The 256^2 result again shows the cost of overhead: CUDA is faster than the CPU, but the margin is much smaller than on the larger images.

Histogram shows the most limited acceleration. For 16,000,000 input integers with bin width 10, runtime decreased from 27.415 ms on the CPU to 14.236 ms with OpenMP and 9.187 ms with CUDA. This is still useful acceleration, but it is much smaller than Count Gliders or Emboss because the work is not a pure map. A very large number of input elements must update a very small number of bins, so both OpenMP and CUDA have to manage many-to-few accumulation. The per-thread OpenMP histograms and per-block CUDA shared histograms reduce global contention, but they do not remove the fundamental serialization pressure at the bin updates and final merge. Figure 3 shows that the CUDA timings vary only modestly with bin width on the large input. Fewer bins reduce the final flush work but increase contention within bins, while more bins spread updates but add more local state. This explains why Histogram does not exhibit the same near-linear scaling trend as the window scan and stencil workloads.

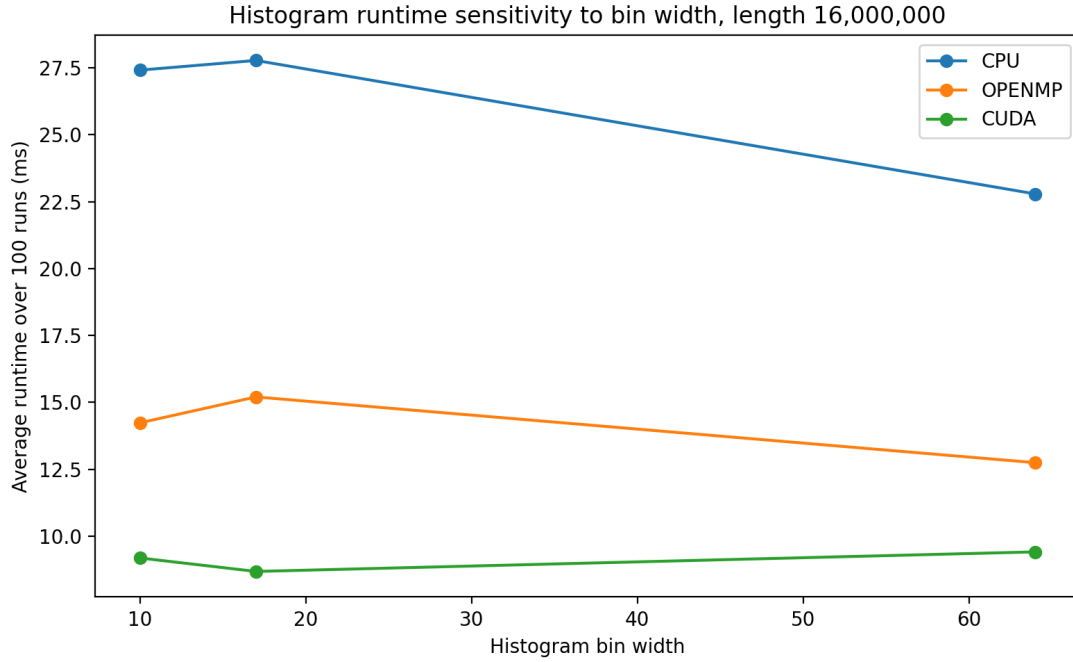


Figure 3: Effect of histogram bin width on runtime for the benchmarked input sizes.

The CUDA profiling evidence in Figure 4 helps separate kernel behaviour from full harness runtime. Nsight Systems measured representative kernel durations of 0.126 ms for Count Gliders, 0.259 ms for Histogram, and 0.207 ms for Emboss on the large profiling cases. These values are shorter than the corresponding benchmark runtimes because the benchmark measures the full wrapper call, including allocation, host-device copies, kernel launch overhead, result copies, and validation-facing synchronisation. This difference is particularly important for interpreting CUDA results: the kernels themselves are short on the large inputs, but the assignment interface does not batch multiple calls or reuse device buffers across the 100 repetitions. The profiling data therefore supports the qualitative explanation of the results without replacing the benchmark data as the main performance metric.

Nsight Systems CUDA kernel timing summary

Kernel	Profile case	Launches	Total ms	Avg us	Min us	Max us
countGlidersKernel(const unsigned int *, unsigned int *)	2048x2048 tiled glider image	1	0.1264	126.430	126.430	126.430
histogramKernel(const int *, unsigned int *)	16,000,000 random ints, bin width 10	1	0.2592	259.227	259.227	259.227
embossKernel(const unsigned char *, unsigned char *)	2048x2048 deterministic RGB	1	0.2066	206.588	206.588	206.588

Figure 4: Nsight Systems kernel timing summary for representative large CUDA inputs.

Overall, the results match the expected parallel patterns. Count Gliders benefits from independent fixed-size classification plus cheap reduction. Emboss benefits from a two-dimensional data-parallel stencil with coalesced output writes. Histogram improves, but its speedup is limited by atomic updates and aggregation into a small result array. The comparison also shows why correctness-preserving algorithmic restructuring matters: the mask-based glider check changes the cost per window without changing the definition of a match, while the histogram aggregation changes where contention occurs without changing the final bins.

5 Limitations and Conclusion

The main limitation of the evaluation is that low-level CUDA hardware counters were not available on the test machine. Nsight Compute reported `ERR_NVGPUCTRPERM`, so the report does not claim measured occupancy, cache hit rate, or memory throughput. The Nsight Systems timings are still useful for observing kernel duration, but they cannot replace a full architectural profile. A second limitation is that the CUDA wrappers intentionally allocate and free memory on each call. This is simple, robust, and compatible with the harness, but it means the reported CUDA benchmark times include overhead that a larger application might reduce by reusing buffers, using pinned host memory, or batching multiple operations.

There are also algorithm-specific limits. Count Gliders is highly parallel, but very small images are too small to benefit from OpenMP or CUDA startup costs. Histogram is constrained by its many-to-few update pattern; shared-memory aggregation reduces global atomic pressure, but bin contention remains the dominant issue. Emboss is straightforwardly data parallel, although neighbouring threads reload overlapping input pixels. A more complex CUDA version could use shared-memory tiles with halo cells, but for this small stencil the added synchronisation and indexing complexity would need to be justified by profiling on larger images.

In conclusion, the submitted OpenMP and CUDA implementations preserve the supplied CPU reference behaviour while applying appropriate parallel patterns for each workload. OpenMP provides clear shared-memory acceleration through reductions, private aggregation, and collapsed loops. CUDA provides larger speedups when the input exposes enough independent work, especially for Count Gliders and Emboss. The weaker Histogram result is not a failure of correctness or implementation alone, but a consequence of the algorithm's contention-heavy structure. The final results therefore demonstrate both effective parallelisation and an understanding of where parallel hardware helps most.

A Project Workflow

The project was completed by first checking the assignment brief, starter headers, CPU reference implementation, and marking requirements to resolve the expected algorithms and interfaces. The code was then implemented only in `openmp.c` and `cuda.cu`, with repeated validation against the supplied CPU version. After correctness checks, benchmark inputs were prepared to cover small, medium, and large cases for each algorithm. The final report was written after collecting Release-mode benchmark results and CUDA kernel timing evidence, so that the explanation was based on observed behaviour rather than implementation intention alone.

B AI Assistance Statement

I used AI assistance during this assignment as a programming, translation, and writing support tool. My own contribution included interpreting the project requirements, deciding the implementation direction, reviewing and accepting the code changes, running the build and validation tests, selecting the benchmark evidence, and checking the final report against the marking criteria. AI assistance was used to help structure the work, suggest implementation patterns consistent with the module content, diagnose build or tooling issues, summarise benchmark data, translate between Chinese and English, and polish the English expression of the report. As English is not my first language, this support helped me communicate my technical reasoning more clearly. The final submitted files remain my responsibility.